APPENDIX D

Glossary

Terms are listed alphabetically; symbols come first. The chapter in parentheses is where the term is introduced or explained in depth.

D.1 Symbols

`'_` — the anonymous lifetime: "a lifetime exists here, I do not need to name it". Its meaning depends on its position: a fresh lifetime in a parameter, the elided output lifetime in a return type, an inference variable in a body. Two occurrences are never forced to be equal. (Chapter 6)

`'1`, `'2`, … — names that rustc invents in diagnostics for lifetimes that have no name in the source, such as the lifetime of an elided `&str` parameter. (Chapter 1)

`'a: 'b` — "`'a` outlives `'b`": the region `'a` contains every point of `'b`. Every lifetime outlives itself. (Chapters 1 and 4)

`'static` — the region that contains every point of the program. `&'static T` is a reference to data that lives until the program ends; the bound `T: 'static` says that `T` contains no borrow that can expire, which every owned type satisfies. (Chapter 7)

`for<'a>` — a binder: "for all `'a`". In a bound, it makes the bound higher-ranked. (Chapter 9)

`T: 'a` — "`T` outlives `'a`": every reference and lifetime parameter inside `T` outlives `'a`, so values of type `T` remain valid throughout `'a`. (Chapters 1 and 7)

`use<..>` — precise capturing: the list of generic parameters, lifetimes included, that an `impl Trait` type may capture. Stable since Rust 1.82. (Chapter 8)

D.2 A–D

Argument-position `impl Trait` (APIT) — `fn f(x: impl Trait)`, an anonymous type parameter. It cannot be named, so it cannot be listed in `use<..>`. (Chapter 8)

Aliasing XOR mutation — the rule behind borrow conflicts: at every point, a value may have any number of shared borrows or exactly one mutable borrow, but not both. (Chapter 1)

`AsyncFn`, `AsyncFnMut`, `AsyncFnOnce` — the traits of async callables (Rust 1.85). Unlike `F: Fn(&T) -> Fut`, a bound `F: AsyncFn(&T) -> U` lets the returned future borrow the argument. (Chapters 9 and 11)

Borrow — the creation of a reference to a place: shared (`&x`) or mutable (`&mut x`). The borrow is *live* while the reference, or anything derived from it, may still be used. (Chapter 1)

Borrowing view — an associated type with a lifetime, such as `type View<'a> where Self: 'a`, that some implementations fill with a borrow and others with an owned value. (Chapter 10)

Borrow checker — the part of the compiler that checks, function by function, that no borrow outlives its value and that no live borrow conflicts with another access. Since 2018 it is based on non-lexical lifetimes. (Chapter 1)

Borrowed forever — the effect of a mutable borrow annotated with the lifetime of the borrowed value's own contents, `&'a mut T<'a>`: because `&mut` is invariant, the borrow must last as long as the value, which can never be used again. (Chapters 2, 3, 4 and 14)

Closure-scoped borrow — an API that lends a reference to a callback instead of returning it (`LocalKey::with`, `thread::scope`), so that the reference cannot escape the call. (Chapters 7 and 12)

Conflict — an access (a borrow, read, write, assignment, move or drop) that is incompatible with a live borrow of the same place. (Chapter 1)

Constant promotion — taking `&` of a constant expression with no destructor and no interior mutability (`&42`, `&[1, 2, 3]`) stores the value in static memory and gives a `&'static` reference. (Chapter 7)

Contract — the signature of a function, seen as a promise: the body is checked against it, and callers are checked against it and nothing else. (Chapter 1)

Contravariant — a type constructor is contravariant in a parameter if subtyping passes through it reversed: `fn(&'short str)` is a subtype of `fn(&'long str)`. Function arguments are the main example. (Chapter 4)

Control-flow graph — the body of a function seen as a graph whose nodes are program points and whose edges lead from each point to every point that can run next. Regions are sets of its points. (Chapter 1)

Covariant — a type constructor is covariant in a parameter if subtyping passes through it in the same direction: `&'long T` is a subtype of `&'short T`, and so is `Vec<&'long T>` of `Vec<&'short T>`. (Chapter 4)

`Cow<'a, B>` — "clone on write": a value that is either borrowed (`&'a B`) or owned; a common alternative to choosing one at the type level. (Chapter 14)

Default object lifetime bound — the lifetime bound the compiler writes for a trait object when you leave it out: `Box<dyn Trait>` means `Box<dyn Trait + 'static>`, `&'a dyn Trait` means `&'a (dyn Trait + 'a)`. (Chapter 8)

Drop check — dropping a value whose type implements `Drop` counts as a use of every borrow the value holds, so those borrows must be live until the drop. (Chapters 5 and 13)

D.3 E-L

Early-bound lifetime — a lifetime parameter that is fixed when a function is *named*, like a type parameter; it happens when the lifetime appears in a where clause or bound, or only in the return type. Function items are not higher-ranked over early-bound lifetimes. (Chapter 9)

Elision — the fixed, syntactic rewrite that fills in the lifetimes you leave out of a signature: a fresh lifetime for each elided input; the input's lifetime for the outputs when exactly one parameter has exactly one lifetime; otherwise the lifetime of the `self` reference. It never looks at the body. (Chapter 3)

Erasure — lifetimes are removed before code generation; they have no run-time representation and cannot be inspected by a running program. (Chapter 1)

Executor — the part of an async runtime that stores futures (*tasks*) and polls them; *spawning* a future hands it to an executor, which is why spawned futures must usually be `Send + 'static`. (Chapter 11)

Generic associated type (GAT) — an associated type with its own generic parameters, such as `type Item<'a> where Self: 'a;`. Stable since Rust 1.65; it makes lending iterators expressible. (Chapter 10)

Guard — a value that borrows a cell or a lock and releases it when dropped: `Ref`, `RefMut`, `MutexGuard`, `RwLockReadGuard`, `RwLockWriteGuard`. (Chapter 12)

Helper-trait pattern — a workaround for higher-ranked bounds over GATs: the lifetime becomes a parameter of a separate trait, with a defaulted parameter whose implied bound restricts `for<'this>` to lifetimes that `Self` outlives. (Chapter 10)

Hidden lifetime parameter — a lifetime parameter of a type that is left out of a path, as in `SplitWhitespace` for `SplitWhitespace<'_>`. Legal, but flagged by `mismatched_lifetime_syntaxes` when it hides a borrow in a return type, and by the allow-by-default `elided_lifetimes_in_paths` everywhere. (Chapter 6)

Hidden type — the concrete type behind an `impl Trait` return type, chosen by the function's body and invisible to callers. (Chapter 8)

Higher-ranked trait bound (HRTB) — a bound that contains a binder, such as `F: for<'a> Fn(&'a str) -> usize`: it must hold for every lifetime, and each use picks the lifetime it needs. `Fn(&str) -> usize` is already one. (Chapter 9)

Higher-ranked type — a type that binds its own lifetimes, such as the function pointer type `for<'a> fn(&'a str) -> &'a str`. It is a subtype of each of its instances. (Chapters 4 and 9)

`#[may_dangle]` — an unstable, `unsafe` promise on a `Drop` impl that the destructor does not use values of a type parameter, which relaxes the drop check; used by `Box`, `Vec` and `HashMap`. Also called the *drop check eyepatch*. (Chapter 13)

Implied bound — an outlives relation that follows from the types in a signature and need not be written: `&'a T` implies `T: 'a`, and `&'a &'b str` implies `'b: 'a`. (Chapter 1)

Interior mutability — mutation through a shared reference, built on `UnsafeCell`: `Cell`, `RefCell`, `Mutex`, atomics. (Chapter 12)

Invariant — a type constructor is invariant in a parameter if no subtyping passes through it: `&mut T` and `Cell<T>` are invariant in `T`, because they allow writes. (Chapter 4)

Late-bound lifetime — a lifetime parameter that is chosen at each call; a function item is higher-ranked over its late-bound lifetimes. Late-bound lifetimes cannot be given with a turbofish (E0794). (Chapter 9)

Leaking — turning an owned value into a reference that lives until the end of the program by never freeing it (`Box::leak`, `String::leak`). Safe, but the memory is never reclaimed. (Chapters 7 and 13)

Lending iterator — an iterator whose items borrow from the iterator itself (for example, overlapping mutable windows into a buffer). It cannot implement `Iterator`; it can be written with a GAT. Also called a *streaming iterator*. (Chapters 2 and 10)

Lifetime — in the borrow checker's model, a region of a function body; in the source, a name such as `'a` (or an elided position) that stands for one. (Chapter 1)

Lifetime parameter — a lifetime declared on a function, type, trait or `impl` (`<'a>`). On a function, it is chosen by the caller at each call and outlives the whole body. (Chapters 1 and 9)

Lifetime soup — a design in which lifetime parameters were added one at a time to silence errors, so that nobody designed the relationships between them. (Chapter 14)

Live — a borrow is live at a point if the reference, or anything derived from it, may be used later along some path from that point. (Chapter 1)

D.4 N–R

Non-lexical lifetimes (NLL) — the borrow checker introduced with the 2018 edition (Rust 1.31): a borrow's region is computed from the uses of the reference, not from the blocks of the source text. (Chapters 1 and 5)

Miri — an interpreter for Rust's intermediate representation, run on nightly Rust, that detects many kinds of undefined behavior in `unsafe` code. (Chapter 13)

NLL problem case #3 — a pattern that NLL rejects although it is safe: returning a borrow on one path and mutating the borrowed value on another. Polonius accepts it. (Chapter 5)

Opaque type — the type behind `impl Trait` in return position: callers know only the traits it implements and the generic parameters it captures. (Chapter 8)

Outlives — see `'a: 'b` and `T: 'a`.

`Pin` / `Unpin` — `Pin<P>` promises that the value behind the pointer `P` will not move again, which self-referential futures need; `Unpin` marks types for which moving after pinning is harmless. (Chapter 11)

`PhantomData<T>` — a zero-sized marker field that makes a type behave as if it contained a `T`: it uses an otherwise unused lifetime or type parameter and determines the type's variance in it. (Chapters 4 and 13)

Polonius — the location-sensitive reformulation of the borrow checker that accepts programs such as NLL problem case #3. On by default in nightly Rust ("Polonius Alpha") since August 2026; check its current status. (Chapter 5)

Over-capturing — an `impl Trait` return type that captures a lifetime its hidden type never uses, which keeps the corresponding argument borrowed for as long as the result lives; the edition 2024 default can cause it, and `use<..>` fixes it. (Chapter 8)

Precise capturing — see `use<..>`.

Raw pointer — `*const T` or `*mut T`: a pointer with no lifetime and no borrow checking; dereferencing it is `unsafe`. (Chapter 13)

Return type notation — the unstable syntax `T::method(..): Send` for bounding the future returned by a trait's `async fn`. (Chapter 11)

Reborrow — a new reference to the place an existing reference points to (`&mut *r`, or implicitly at a call), with a region that may be shorter. While a mutable reborrow is live, the original reference is frozen. (Chapter 5)

Receiver — the `self` parameter of a method, in any of its forms (`self`, `&self`, `&mut self`, `self: Box<Self>`, `self: Pin<&mut Self>`, ...). (Chapters 2 and 3)

Region — a set of points in the control-flow graph of a function body: the borrow checker's notion of a lifetime. (Chapter 1)

Return-position `impl Trait` (RPIT) — `fn f() -> impl Trait`, which returns an opaque type. In edition 2024 it captures every generic parameter in scope, lifetimes included, unless `use<..>` says otherwise. In traits it is called RPITIT, and it captures every parameter in all editions. (Chapter 8)

D.5 S–Z

Self-referential value — a value that holds a reference to its own data, such as a future that keeps a reference to one of its locals across an `.await`. Safe Rust cannot build or move one without `Pin`; ordinary code stores indices instead. (Chapters 11 and 13)

Scope of a value — from the creation of a value until it is dropped or moved away. A borrow's region must fit inside it. (Chapter 1)

Scope of a variable — the block in which a variable is declared; a syntactic notion that, since NLL, does not determine how long a borrow held by the variable is live. (Chapter 1)

Scoped thread — a thread spawned through `thread::scope`, which is joined before `scope` returns and may therefore borrow from the enclosing function. (Chapter 7)

Scrutinee — the expression a `match`, `if let` or `while let` examines. Temporaries created in it can live longer than you expect. (Chapters 5 and 12)

Soundness — a safe API is sound if no safe code can use it to cause undefined behavior, whatever the arguments. (Chapter 13)

Subtype — `S` is a subtype of `T` if a value of type `S` can be used wherever a `T` is expected. For lifetimes, `&'long T` is a subtype of `&'short T` when `'long: 'short`. (Chapter 4)

Tail expression — the final expression of a block, without a semicolon, which gives the block its value. Edition 2024 drops its temporaries before the block's local variables. (Chapters 5 and 12)

Temporary — an unnamed value that the compiler creates to evaluate an expression, such as the result of a function call that is borrowed immediately. It is dropped at the end of its *temporary scope*, usually the end of the statement. (Chapter 5)

Temporary lifetime extension — a temporary borrowed in certain positions of a `let` initializer (`let r = &make();`) lives until the end of the enclosing block instead of the end of the statement. (Chapter 5)

Two-phase borrow — a mutable borrow that is first *reserved*, acting as a shared borrow, and *activated* only when the call begins; it is why `v.push(v.len())` compiles. (Chapter 5)

Unbounded lifetime — a lifetime that nothing constrains, as when a raw pointer is turned into a reference: it can become any lifetime the caller wants, including `'static`. Bound it with a signature. (Chapter 13)

Universal region — rustc's name for a lifetime parameter of the function being checked: it is chosen outside the function and contains every point of the body. (Chapter 9)

Unsizing coercion — the conversion of a pointer to a concrete type into a pointer to a dynamically sized type, such as `Box<Circle>` to `Box<dyn Shape>` or `&[T; N]` to `&[T]`. When the target is a trait object, the object's lifetime bound must be satisfied by the concrete type. (Chapter 8)

Waker — the handle a pending future uses to ask to be polled again; `Waker::noop()` (Rust 1.85) does nothing and is enough for the book's minimal `block_on`. (Chapters 9 and 11)

Variance — how subtyping between lifetimes (or types) passes through a type constructor: covariant, contravariant or invariant. (Chapter 4)

Well-formed type — a type whose implied requirements hold: `&'a T` is well-formed only if `T: 'a`. The compiler assumes that the types in a signature are well-formed, which is where implied bounds come from. (Chapter 1)

Witness function — a function whose body only returns its argument at a different type, such as `fn f<'s>(x: &'static str) -> &'s str { x }`; it compiles exactly when the subtyping holds. (Chapter 4)

Zero-copy parsing — parsing in which the results (tokens, fields, entries) are slices of the input, so that producing them allocates nothing. (Chapter 14)